LA_pack primer

Abstract

GNU APL implements the APL functions monadic ⌹ (aka. **Matrix Inverse**) and dyadic ⌹ (aka. **Matrix Divide**) by means of *LApack functions*. These LApack functions were originally written in FORTRAN, but then manually translated from FORTRAN to C++ by the author. In this translation, features of the LApack functions that were not needed for ⌹ were removed in order to simplify the C++ code as much as possible.

Monadic ⌹: Matrix Inverse

Monadic ⌹ is the special case of dyadic **A⌹B** where A is the unit matrix:

```
Let A←(↑⍴B) ∘.= (↑⍴B). Then ⌹B ⟷ A⌹B.
```

Bif_F12_DOMINO::eval_B(Value_P B) const, i.e. the implementation of monadic ⌹, simply constructs a unit matrix **I** of proper size and then returns **eval_AB(I, B)**, i.e. the result of the implementation of dyadic **A⌹B**.

Dyadic A⌹B: Matrix Divide

In APL, **A⌹B** for a N-column matrix **A** is defined im terms of each column of **A**:

```
If  X←A⌹B  then  X[;J] ⟷ A[;J]⌹B for all (scalar) columns J∈⍳¯1↑⍴A.
```

In other words, computing **A⌹B** for a matrix **A** boils down to computing **A⌹B** for (column-) vectors (which are the columns of **A**). For every such column vector a of **A**, the result is a column vector **x** of **X**:

```
B ∘ x ≡ a   ⟷   x ≡ a ⌹ B   ⟷   x ≡ a ∘ B⁻¹
```

In GNU APL: **A +.× B ←→ A ∘ B**. The vast majority of inner products in real-life APL programs is used for matrix multiplication aka. **A +.×B**. The GNU APL function **A∘B** is an optimized version of the special case **A +.× B** of the more general inner product **A f.g B**.

Since **B** is the same for all columns a of **A** (and their corresponding result columns **x** of **X**), is the result **X** of **X←A⌹B** computed in two steps:

1. compute **B⁻¹**
2. compute **X←A∘B⁻¹**

The second step **X←A∘B⁻¹** is simple, therefore only the first step, the computation of **B⁻¹** is of concern here.

BTW. the iteration over the columns of **A** in the second step above occurs near the end of function **LA_pack::gelsy()** in function **trsm<T>()**.

In order to compute $\mathbf{B^{-1}}$, matrix **B** is factorized into two matrices **Q** and **R** which have the following properties:

- **Q** is orthogonal (resp. Hermitian for complex **B**). That is: $\mathbf{Q^{-1} \equiv \backslash + \lozenge Q}$,
- **R** is upper triangular, i.e. $\mathbf{R_{ij} = 0}$ for all $\mathbf{i > j}$,
- $\mathbf{B = Q \circ R}$,

Once **Q** and **R** are found, $\mathbf{B^{-1}}$ can be computed easily with:

$$B^{-1} = (Q \circ R)^{-1} = R^{-1} \circ Q^{-1} = R^{-1} \circ \lozenge Q.$$

The inverse of an upper triangular matrix (such as R^{-1}) is far easier to compute than the inverse of a general matrix (such as B^{-1}). See below. What remains is therefore the computation of a QR factorization of an arbitrary matrix B.

Inversion of an orthogonal Q

The inversion of an orthogonal **Q** matrix **Q** means transposition of **Q**. However, unlike in APL where **⍉Q** is a new value computed from **Q**, this transposition is never computed in C++ or in FORTRAN. Instead, where **⍉Q** is needed, the matrix **Q** is simply accessed with the row and column interchanged. IOW, with a little care as to what are rows and what are columns is ⍉Q no-op. In GNU APL three typedefs are used to distinguish rows and columns:

```
In LApack.hh:

typedef int Crow;   // row number
typedef int Ccol;   // column number
typedef int Cdia;   // diagonal item (row == column)
```

The translation from FORTRAN to C++ is somewhat tricky for two reasons:

1. FORTRAN indices run from 1..N while C++ indices run from 0..N-1
2. FORTRAN matrices are stored in column-major order (i.e. adjacent items in memory belong to the same column) while C++ matrices are stored in row-major order (i.e. adjacent items in memory belong to the same row).

Inverting an upper trianguar matrix R

Suppose $\mathbf{R = op_1 \circ op_2 \circ \ldots \circ op_n \circ ID}$ where every $\mathbf{op_k}$ adds a multiple of some row of R to some other row of R. Then, as we learn in school, $\mathbf{R^{-1} = ID \circ op_1 \circ op_2 \circ \ldots \circ op_n}$. In other words, if the operations $\mathbf{op_1}$, $\mathbf{op_2}$, …, $\mathbf{op_n}$ turn a matrix **R** into the identity matrix, then the same operations (in the same order) turn the the identity matrix into the inverse $\mathbf{R^{-1}}$ of **R**.

The operations that turn an upper trianguar matrix **R** into the identity matrix ID are fairly simple:

1. **R** is upper triangular. Therefore the last row of **R**, say row **n**, has only one nonzero element r_{nn} on its diagonal. For every row above that last row, say row **k** (i.e. with k < n), do:
    1. Let r_{kn} be the last element in row **k**. Subtract a multiple of the last row from row **k** so that r_{kn} becomes **0**.
    2. That multiple is $r_{kn} \div r_{nn}$.
    3. Since all other elements of the last row are **0**, this only sets r_{kn} to **0** but does not change any other element of row **k**.
    4. After repeating this step for every k < n was the entire last column of the matrix **R** is **0**, except for the element r_{nn} on the diagonal.
2. Repeat the same step for the second-last, third-last, … first column of the matrix. In every iteration one column of **R** is set to 0 above the diagonal. After the last **k=1** the matrix is diagonal (i.e. all elements above **and** below the diagonal are **0**).
3. Finally, subtract a multiple of every row from itself so that the diagonal item becomes **1.0**.
    1. That multiple for diagonal element r_{kk} of row k is $(1-r_{kk}) \div r_{kk}$.
    2. After repeating that step for every **k ≤ n** is **R** the unit matrix.
4. Notice that every operation above subtracts a multiple of some row from some other (or the same) row.

The algorithm above defines the sequence $\mathbf{op_1 \circ op_2 \circ \ldots \circ op_n}$ above, where **n** is the number of columns in **R**. In practice (and in LApack and consequently also in GNU APL) these operations are not performed on **R** first and the on the identity matrix afterwords but rather in parallel:

1. Start with a matrix **R, AUG** where **AUG←(⍴R)↑(⍳↑⍴R)∘.=⍳↑⍴R)**. That is, **AUG** is initially the unit matrix with the same shale as **R**. The matrix **AUG** (or sometimes the matrix **R,AUG**) is commonly known as the *augmented matrix*.
2. The operations described above are then simultaneously applied to the left half (i.e. to **R**) and to the right half (i.e. to **AUG**) until the left half **R** of **R,AUG** has become the unit matrix. At that point, the right half **AUG** has become the desired inverse $\mathbf{R^{-1}}$ of **R**.
3. All this happens in functions **LA_pack::invert_UTM()** and **LA_pack::invert_QUTM()**. The term **QUTM** stands for **Q**uadratic **U**pper **T**riangular **M**atrix and takes advantage of the fact that the inversion of a non-quadratic upper triangular M×N matrix is essentially the inversion of its upper N×N submatrix. The rows N+1..M are all zero and become zero columns in the inverse.

QR factorization

Householder matrix

For the moment we consider only real matrices; the complex case is almost

identical, except that **⍉X** becomes **+⍉X** in the complex case. **\+⍉X** is called the *conjugate transpose* of **X** which is the same as the "normal" transpose if **X** is real.

Let **B** be an arbitrary M×N matrix.

Let **V←1↓B[;1]**, i.e. V is the first column of **B** except the diagonal item **B[1;1]**.

Let **L←+/V×V**, i.e. **L** is (the square of) the Euclidean length of **V**.

The Householder matrix **H=H(B)** for matrix **B** is then defined as:

$$H = I - \frac{2}{L} \times V \circ.\times V.$$

That is:

$$H = (h)_{ij} = \begin{cases} 1.0 - 2\times V[i]\times V[i] & \text{if } i=j \text{ , and} \\ -\ 2\times V[i]\times V[j] & \text{otherwise.} \end{cases}$$

It follows immediately that:

1. **H** is *Hermitian* (aka. *symmetric* if **H** is real). I.e. **H = +⍉H**,
2. **H** is *unitary* (aka. *orthogonal* if **H** is real). I.e. **H^{-1} = ⍉H**, and
3. **H** is *involutary*. I.e. **H = H^{-1}**

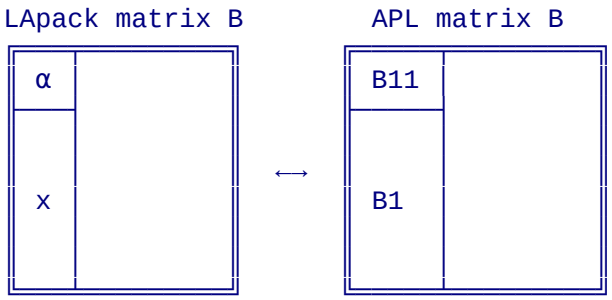
These really nice properties of **H**:

1. make the computation of the transposition **⍉H** essentially a no-op, and
2. ensures that the composition **$H = H_1 \circ H_2 \circ \ldots \circ H_n$** of several Householder matrices is also Hermitian (complex case) resp. orthogonal (real case).

In the following let:

- **B** be an arbitrary real or complex matrix,
- vector **B1** be the first column of **B**, and
- scalar **B11** be the first item of column vector **B1** (= the first item on the diagonal of B).

In the LApack code is **B1** usually a variable named (lowercase) **x** and **B11** a scalar variable named (uppercase) **ALPHA** (or **α** in comments). That is, matrix B looks like this:



There is a subtle but noteworthy differences between the APL code discussed below and the LApack FORTRAN or C++ code:

- In the original FORTRAN code of LApack, the vector **x** *excludes* the diagonal element **α** and **α** is passed as a separate function argument or result.
- In the GNU APL C++ translation of that code, the vector **x** also *excludes* the diagonal element **α**. However, **α** is not passed as a separate function argument like in FORTRAN. Instead C++ takes advantage of the fact that our matrices are stored in FORTRAN order which makes **α ←→ x[-1]** and therefore a function that already knows **x** can easily recover **α** from **x**.
- In the APL code below, the vector **B1** (which corresponds to **x**) *includes* the scalar **B11** (which corresponds to **α**).

Instead of a strict mathematical proof we use APL as a tool of thought.

Let **V** be a real or complex vector. The following APL function named **Householder** simply implements the definition of a Householder matrix given above:

```
∇H←Householder V;I;L
 ⍝
 ⍝ return the Householder matrix (aka. elementary reflector) for vector V
 ⍝
 I←I∘.=I←⍳⍴V         ⍝ I:  the (⍴V)×(⍴V) identity matrix,
 L←+/|V×+V           ⍝ L:  ‖V‖² i.e. the square of the norm of vector V
 H←I-(2÷L)×V∘.×+V    ⍝ H:  the Householder matrix for vector V
∇
```

Next we define a somewhat random matrix **B** that shall prove our claims by example (aka. *incomplete induction*):

```
MN←(M N)←4 3                  ⍝ choose a matrix size (M≥N)
⎕RL←+/⎕TS ◊ B←?(2, MN)⍴18     ⍝ prepare for random B
B←+⍀9 0J9×[1]B                ⍝ some complex M×N random matrix B
B11←↑B1←B[;1]                 ⍝ B1 is column 1 of B, B11 its first item
```

With these prerequisites we can "prove" the following

Proposition: Let

```
L11←×B11              ⍝ B11 normalized to length 1 (= ±1 for real B11)
LB1←(+/B1×+B1)*0.5    ⍝ scalar LB1 is ‖B1‖ i.e. the length of B1

H1←Householder V1←B1 + L11×M↑LB1   ⍝ a Householder matrix (from V1)
H2←Householder V2←B1 - L11×M↑LB1   ⍝ another Householder matrix (from V2)
```

Proofs (by APL examples **B**):

```
∇H←Householder V;I;L
 ⍝
 ⍝ return the Householder matrix (aka. elementary reflector) for vector V
 ⍝
 I←I←I∘.=I←⍳⍴V       ⍝ I: the (⍴V)×(⍴V) identity matrix,
 L←+/|V×V            ⍝ L: ‖V‖² i.e. the square of the norm of vector V
```

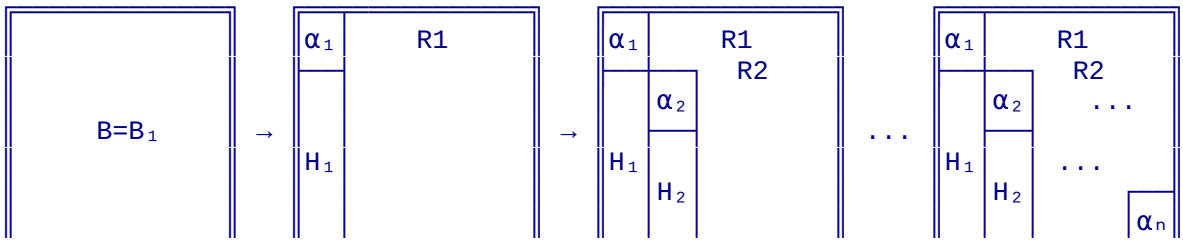
```
      H←I-(2÷L)×V∘.×+V   ⍝ H: the Householder matrix for vector V
     ∇

      LB1←(+/B1×+B1)*0.5   ⍝ scalar LB1 is ‖B1‖ i.e. the length of B1
      L11←×B11             ⍝ B11 normalized to length 1 (= ±1 for real B11)
      H1←Householder B1+L11×M↑LB1
      H2←Householder B1-L11×M↑LB1

      H = +⍉H              ⍝ H is Hermitian
1 1 1
1 1 1
1 1 1

      (⌹H) = ⍉H            ⍝ H is unitary
1 1 1
1 1 1
1 1 1

      H = ⌹H               ⍝ H is involutary
1 1 1
1 1 1
1 1 1

      B
 27J117  54J72   18J27
 81J117  18J117 126J144
117J81  117J108   9J54
135J27  117J18  117J90

      2⍕H1 ∘ B   ⍝ notice the first column
 ¯61.12J¯264.83 ¯33.28J¯233.03  ¯8.67J¯203.23
    .00J.00      ¯52.41J25.80    86.09J69.90
    .00J.00       19.00J47.42   ¯54.44J¯1.31
    .00J.00        6.71J1.84     39.43J65.25

      2⍕H2 ∘ B   ⍝ notice the first column
 61.12J264.83  33.28J233.03   8.67J203.23
   .00J.00     ¯19.79J¯30.51  73.02J¯12.81
   .00J.00      29.70J¯16.77 ¯95.79J¯74.13
   .00J.00      ¯8.43J¯59.27 ¯25.25J16.46
```

Repeated Application of Reflectors

Once we know how to set the first column of some arbitrary matrix **B** to **0** (below the diagonal) we can now turn the entire matrix into an upper triangular form:

1. Set $B_1 = B$ and find a reflector H_1 that sets the items below the first column of B to 0.
2. Repeat that step for $B_2 = 1\ 1\downarrow B_1$, $B_3 = 1\ 1\downarrow B_2$, ... $B_n = 1\ 1\downarrow B_{n-1}$ which computes reflectors a sequence H_1, H_2, ... H_n. The matrices B_k and H_k become smaller and smaller in this process.

After n steps matrix B looks like this:



For lack of space we show **$\alpha_1 \ldots \alpha_n$** on the diagonal above, but in reality the diagonal is **$R_{11}, R_{22}, \ldots R_{nn}$**. The sequence **$\alpha_1 \ldots \alpha_n$** is stored in **tau** as explained above.

Now:

1. R is upper triangular (where the items below the diagonal are 0 by definition and therefore need not be stored in B)
2. **$H \leftarrow H_1 \circ H_2 \circ \ldots \circ H_n$** is unitary resp. orthogonal since it is the product of n unitary resp. orthogonal matrices **H_k**.

The iteration above takes place in function **LA_pack::laqp2()** which starts with matrix **B** and ends with **R** in the upper half (including the diagonal), **Q** in the lower half, and the diagonal factors **$\alpha_1 \ldots \alpha_n$** stored in variable **tau**.

At this point the lower half and **tau** are not the matrix **Q** itself but the reflectors that turn the identity matrix into **Q**. For this reason we usually refer to lower half as **HR** and not as **QR** in the GNU APL C++ code.

Numerical considerations

In mathematics we compute with exact numbers and would be finished at this point. On real computers we work with approximation of exact numbers which may cause problems with the accuracy of the result.

Let **α** and **β** be numbers. For the sake of explanation assume **α>0** and **β>0**. Due to rounding, do **α** and **β** only determine two ranges in which the exact α and **β** lie:

$$\alpha - e_1 \le \text{exact } \alpha \le \alpha + e_1 \text{ and}$$
$$\beta - e_2 \le \text{exact } \beta \le \beta + e_2$$

The (usually small) numbers **e_1** resp, **e_2** define the *absolute error* of **α** resp. **β** while the quotients **$e_1 \div \alpha$** resp. **$e_2 \div \beta$** define the *relative_error* of **α** resp. **β**.

Now, adding or subtracting the inequalities above gives:

$$\begin{array}{rl} & \alpha - e_1 \le \text{exact } \alpha \le \alpha + e_1 \\ \pm & \beta - e_2 \le \text{exact } \beta \le \beta + e_2 \\ \hline & (\alpha \pm \beta) - (e_1 + e_2) \le \text{exact } \alpha \pm \beta \le (\alpha \pm \beta) + (e_1 + e_2) \end{array}$$

This means that the absolute error of the sum or difference **(α±β)** is the sum of the absolute errors of **α** and **β**. This is no problem as long as **(α±β)** does not come too close to **0**. If it does, however, then the absolute error remains small, but the relative error is now:

$$\frac{e_1 + e_2}{\alpha \pm \beta} \to \infty \quad \text{as } (\alpha \pm \beta) \to 0$$

That is, for **(α-β)** can the relative error become huge if **α≈β**. In LApack these issues are addressed in 3 ways:

1. If the input matrices are very large or very small (in terms of their norm) then they (and their result) are scaled up or down so that their coefficients land in a range that is not subject to major rounding error. See function **LA_pack::gelsy()** vs. **LA_pack::scaled_gelsy()**.
2. As we have seen above, for any matrix **B** there are actually two reflectors **H1** and **H2** each of which bring the first column off **B** below the diagonal to **0). LA_pack then chooses the one that produces the largest difference *V1** resp. **V2** above.
3. Some functions, in particular **LA_pack::larfg()** take additional measures to minimize the errors caused by too small arguments.
4. Column pivoting. instead of processing the columns in the order different from B[;1], B[;2], … B[N] as to avoid small coefficients in the subsequent matrices.

The column pivoting works like this:

1. In every iteration above:
    1. Compute the norms of the columns,
    2. Find the column **k** with the largest norm (called "the pivot")
    3. If **k≠1** (i.e. the first column does not have the largest norm already) then exchange columns **1** and **k** before proceeding.
    4. In the context of solving equations, the pivoting is essentially a renaming of the variables $x_1, x_2, \ldots, x_n$. This renaming is later undone (see the end of **scaled_gelsy()**) before the result is returned to APL.

Memory Layout

At the time when FORTRAN was popular, memories were small. The LApack FORTRAN code addressed this in two ways:

Blocked execution

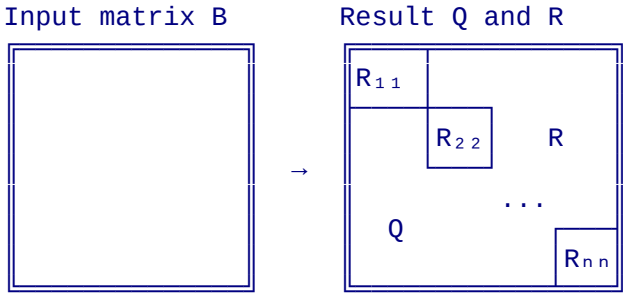
Blocked operation means that the processing of matrices that may not fit into the core memory of a computer were processed in chunks. Some FORTRAN functions therefore had a "blocked" version (for large matrices) and an "unblocked" version (for small matrices and for the chunks of large matrices).

These days memory is large and therefore blocking was removed entirely. Only the unblocked FORTRAN functions survived in the GNU APL code.

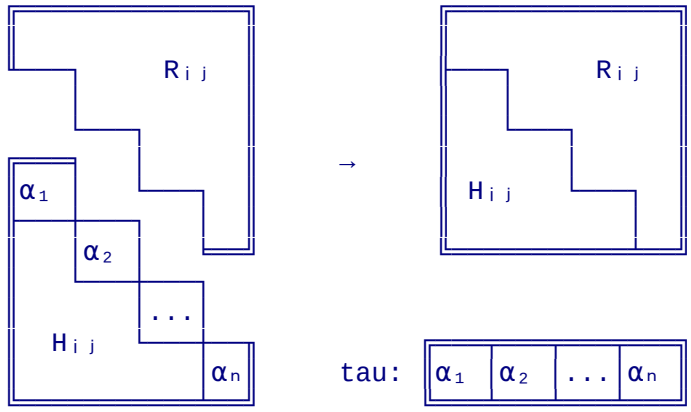
In-place computation

Another heavily used trick to reduce memory consumption is in-place computation. Instead of passing the argument of a function in one variable and storing the result in a another variable, the same variable was used for the largest input variable and

the result. For example, function **laqp2()** comnputes the QR factorization of an arbitrary matrix **B** and the result is stored at the same memory location. However, the result **QR** is split in two pieces along the diagonal, taking advantage of the fact that **Q** is symmetric (so its upper half can be derived directly from its lower half, and that **R** is upper triangular (so its lower half is implicitly **0** and therefore not of interest):



Unfortunately in this scheme the matrices of **Q** and **R** now compete for the former diagonal of input **B**. This is solved by storing in the diagonal of **Q** in a separate vector (named **tau** in the LApack code):



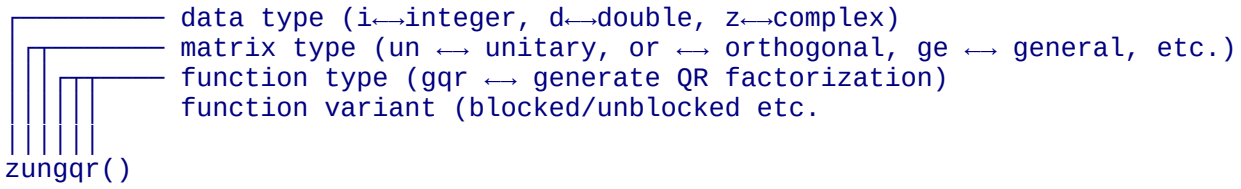
Another issue to understand LApack is that functions **laqp2()** and friends do **not** return the matrices **Q** and **R** of the factorization **B = Q∘R** directly. Instead of **Q** it returns a matrix **H** with the Householder reflectors for the different columns. The matrix **Q** can then be obtained by applying these reflectors to the **n×n** unit matrix **ID(n)**:

```
Q ⟷ H∘ID(n)
```

This avoids one intermediate matrix multiplication. Instead of **Q←H∘I** followed by some other **Z←Q∘R**, LApack leaves **H** for the moment and later computes **Z←H∘R** directly. In addition LApack avoids the computation of the householder matrices used in the APL examples above for demonstration purposes. Instead of full k×k reflector matrices it uses the reflector vectors, typically called **v**, and computes the elements H_{ij} of each reflector as needed. The lifetime of each H_{ij} is usually short, therefore it makes little sense to store it in a large matrix.

LApack function names

The names of LApack functions look somewhat acary at first glance. For example the name of function **zungqr()** may not be too obvious. However these names follow a naming scheme that rougly looks as follows:



With this nameing scheme function **zungqr()** for complex matrices becomes **dorqr2()** for real matrices.

In FORTRAN every complex function (like **zungqr()** has a real counterpart **dorqr2()** for real matrices. In fact there are even more variants because there are different precisions (like 32-bit **float** and 64-bit **double** in C++) and each has its own FORTRAN functions. That made a lot of sense in the FORTRAN days where it made a considerable diference in performance.

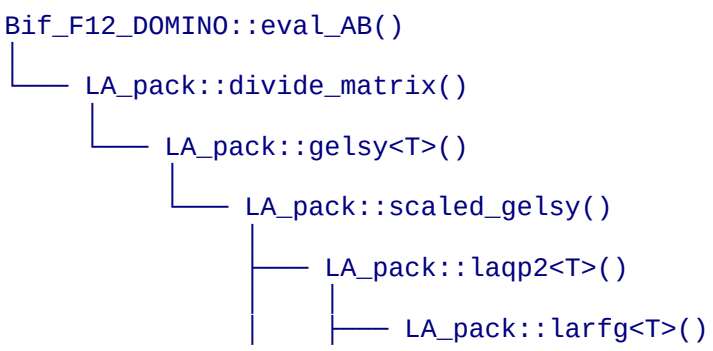
A closer look at these functions revealed, however, that the difference between data types are actually rather small. The GNU APL version of these functions make the data type a template parameter so that every group of FORTRAN functions becomes a single template function in C++. In our example:

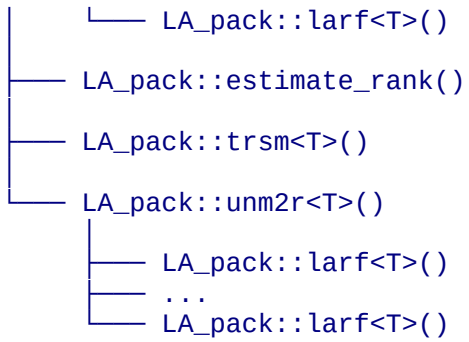
```
FORTRAN:

    dorgqr   QR-factorize real matrix
    zungqr   QR-factorize complex matrix

becomes C++

    template<T>  ungqr()
```

where the template argument **<T>** is either **<DD>** for real or **<ZZ>** for complex numbers. The use of C++ templates cuts the number of functions needed roughly in half,

Call Tree

With all thee in mind, **A⌹B** aka. **Bif_F12_DOMINO::eval_AB** becomes simple. We look at its call tree, which is roughly this:





In this call tree:

- **eval_AB()** performs the APL argument checking (matching shapes of **A** and **B**) and figures if **A** or **B** contain complex items. It also treats scalars resp. vectors **A** and **B** as 1×1 resp. 1×N matrices
- **divide_matrix()** translates the ravels of APL matrices **A** and **B** into (transposed) FORTRAN matrices
- **gelsy<T>()** scales (and later unscales) FORTRAN matrices **A** and **B** with very small or very large items so that subsequent operations are safe in regard to rounding errors. In almost all real life cases **A** and **B** will not need any scaling.
- **scaled_gelsy()** computes **Z←A⌹B** for the now properly scaled **A** and **B**
- **laqp2()** computes a QR factorization **B=Q∘R** of **B**. In the result:
  - **Q** is orthogonal (for real **A** and **B**) or Hermitian (for complex **A** or **B**). **Q** is easy to invert since **Q⁻¹ ←→ +⍉Q**, and
  - **R** is upper triangular (so that **R∘X=C** is simple to solve)
- In the QR factorization: for every column of **B**:
  - **larfg<T>()** generates one reflector (i.e. for one column of **B**)
  - **larf<T>()** applies that reflector to a submatrix of **B**
- **estimate_rank()** checks that the system **A=B∘X** of linear equations is not under-determined and raises a DOMAIN ERROR if it is. In contrast, over-determination (more equations than variables) is accepted and produces a least-square solution for **A=X=B∘X**
- **trsm<T>()** computes **R⁻¹** for the upper triangular factor **R** of **B=Q∘R** that was computed above
- **unm2r<T>()** applies **R⁻¹** to **A**

⌹ with axis: QR factorization

The first non-trivial thing that happens in monadic **⌹B** and in dyadic **A⌹B** is a *QR-factorization* of matrix **B**. A QR factorization of a matrix B is a pair **(Q R)** of matrices with the following properties:

1. **Q** is orthogonal (resp. Hermitian if B is complex),

2. **R** is upper triangular, and
3. **B ≡ Q ∘ R**

The same matrix **B** may have different QR factorizations with the above properties. And different factorization algorithms may produce different pairs **(Q R)** for the same matrix **B**.

Often the orthogonal **Q** and/or the upper triangular **R** of a factorization **B=Q∘R** is more valuable than the final result **B⁻¹=R⁻¹∘Q⁻¹** of ⌹**B** or **A**⌹**B**. For this reason GNU APL provides ⌹**[X]** as to obtain the matrices **Q** and **R** without applying them (i.e. in order to solve **A∘X=B**). As a matter of convenience for the programmer, GNU APL not only computes the pair **(Q R)**, but a triple **(Q, R, Rinv)** where **Rinv ≡** ⌹**R**. One could, of course, compute **Rinv** from **R** in APL with ⌹. But **Rinv** is anyhow computed in the QR factorization of **B** and therefore it is more efficient to return it together with **R** rather than computing it after **R** (which would QR-factorize **B** one more time).

GNU APL has implemented two different algorithms for the QR-factorization of a matrix **B**, which produce slightly diffferent results.

1. The first algorithm is based on a factorization algorithm publiashed by **Garry Helzer** in **APL Quote Quad, Volume 9, Issue 3, 1979**, and
2. the second algorithm is, like monadic and dyadic ⌹, based on LApack function **laqp2()** but with column privoting disabled.

The algorithm to be used is selected with axis argument **X** as follows:

1. Integer scalar 1 selects Garry Helzer's algorithm,
2. Integer scalar 2 selects the laqp2() algorithm, and
3. (obsolete) a small real scalar like ⎕CT also selects Garry Helzer's algorithm. This is for backward compatibility with older GNU APL versions (that always used Garry Helzer's algorithm).

For quadratic matrices **B**, i.e. for **ρB ←→ N N**, are the results of the two algorithms, apart from rounding errors, identical.

For non-quadratic matrices **B** with shapes **ρB ←→ M N** with M > N, the results differ as follows:

	Helzer	**laqp2()**	**Notes**
axis X	1	2	
ρQ	M M	M N	rows N+1 .. M non-zero
ρR	M N	N N	rows N+1 .. M padded with 0
ρRinv	N M	N N	columns N+1 .. M padded with 0
ρQ∘R	ρB		
N N↑R	Same		

	Helzer	laqp2()	Notes
N N↑Rinv	Same		
(⍉Q)∘Q	ID N		ID N ←→ (⍳N)∘.=(⍳N)
Q∘(⍉Q)	ID N	≠ ID N	
Rinv∘R	ID N		
R∘Rinv	M M↑ID N	ID N	

For bread and butter applications is Helzer probably the alternative that is simpler to use. Lapack puts quite some effort into performance optimizations and numerical border cases. Therefore the LApack alternative might be better suited for large matrices or when precision should become an issue.

Last updated 2023-09-05 16:20:53 CEST